

Dynamic Programming

Satoshi Fukuda

Swiss Program for Beginning Doctoral Students in Economics 2026

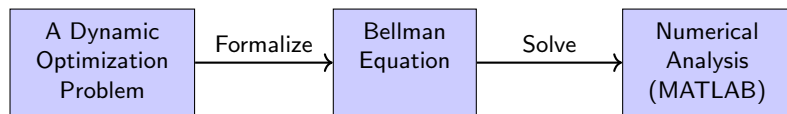
August 6, 2026

Dynamic Optimization Problems

There are many dynamic economic problems!

- Macroeconomics, International Economics, and Finance
 - ▶ Consumption, Investment, Growth, Monetary Policy, ...
 - ▶ Search, ...
 - ▶ Asset Pricing, Corporate Finance, ...
 - ▶ Trade, ...
- Health, Labor, Development, Education, Family Economics
 - ▶ Job Search, Human/Health Capital, Self-Insurance, ...
- IO and Micro Theory
 - ▶ Firm Dynamics (Entry/Exit), R&D, Pricing, Durable Goods, Contracting, Dynamic Decision Making (including Behavioral applications) ...
- Public, Political Economy
 - ▶ Optimal Taxation, Sovereign Debt, Institutional Dynamics, ...
- Environment
 - ▶ Resource Management, Climate Change, ...

Outline



① Theory of Discounted Dynamic Programming

- ① Formulation of a Dynamic Optimization Problem
- ② Bellman Principle of Optimality
- ③ Solving a Bellman Equation
 - ★ First-Order Condition and Envelope Formula
 - ★ Contraction Mapping Theorem and Value Function Iteration

② MATLAB Numerical Implementation

- ① Cake-Eating Problem (Dynamic Consumption)
- ② Ramsey Growth Model: Optional
- ③ Lucas Asset-Pricing Model (As long as time permits)

Dynamic Optimization Problem

- Time is discrete and horizon is infinite.
- We maximize the **discounted sum** of a (concave) reward function $r : \mathbb{R}^n \times \mathbb{R}^k \rightarrow \mathbb{R}$, subject to **some constraints**, by **choosing a sequence of “controls”** $(u_t)_{t=0}^\infty$:

Sequential Optimization Problem

$$\max_{(u_t)_{t=0}^\infty} \sum_{t=0}^{\infty} \beta^t r(x_t, u_t)$$

subject to $x_{t+1} = g(x_t, u_t)$ for each $t \geq 0$ and $x_0 \in \mathbb{R}^n$ is given.

- ① Choosing $(u_t)_{t=0}^\infty$ instead of (u_0, u_1) .
- ② Yet, we consider “today vs. tomorrow on” (how?)
- Note: To focus on the key ideas, we implicitly assume that the problem is well-defined.

Dynamic Optimization Problem

Sequential Optimization Problem

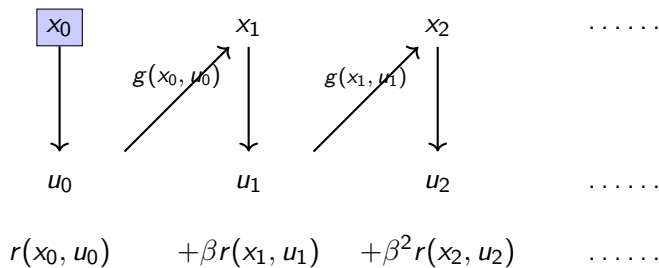
$$\max_{(u_t)_{t=0}^{\infty}} \sum_{t=0}^{\infty} \beta^t r(x_t, u_t)$$

subject to $x_{t+1} = g(x_t, u_t)$ for each $t \geq 0$ and
 $x_0 \in \mathbb{R}^n$ is given.

We want to choose an optimal sequence $(u_t)_{t=0}^{\infty}$
under the following constraint at each time $t \in \mathbb{N}_0 := \mathbb{N} \cup \{0\}$.

- The value of the *state variable* $x_t \in \mathbb{R}^n$ is given.
- With x_t given, we choose a *control* (variable) $u_t \in \mathbb{R}^k$.
- With (x_t, u_t) determined, the next period's state variable x_{t+1} is determined through the *transition equation* $x_{t+1} = g(x_t, u_t)$.

Dynamic Optimization Problem



Dynamic Optimization Problem

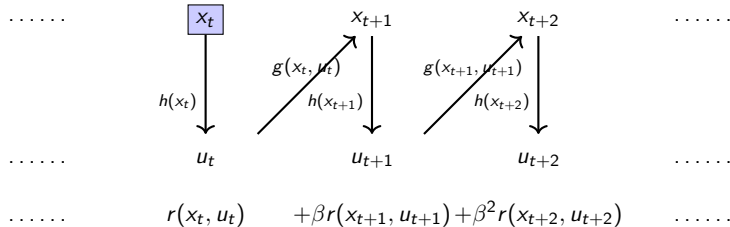
The Value Function

$$V(x_0) := \max_{(u_t)_{t=0}^{\infty}} \sum_{t=0}^{\infty} \beta^t r(x_t, u_t)$$

subject to $x_{t+1} = g(x_t, u_t)$ for each $t \geq 0$ and $x_0 \in \mathbb{R}^n$ is given.

- Starting with the initial value x_0 , the value $V(x_0)$ of the maximization problem is defined (given that the problem is well-defined).
- We call V to be the *value function*.
- Example. x_0 : wealth today $\implies (u_t)_{t=0}^{\infty}$: consumption path $\implies V(x_0)$: the discounted utility

Dynamic Optimization Problem



- The problem is stationary in the sense that

$$V(x_t) = \beta^{-t} \max_{(u_\tau)_{\tau=t}^{\infty}} \sum_{\tau=t}^{\infty} \beta^\tau r(x_\tau, u_\tau) \text{ subject to the constraints.}$$

- Given stationarity, denote next period's variables by adding prime (') instead of time subscripts.

Today	→	Tomorrow
x_t	→	x_{t+1}
x	→	x'
- A function $h : \mathbb{R}^n \ni x \mapsto h(x) = u \in \mathbb{R}^k$ is called a *policy* function.

Bellman Principle of Optimality

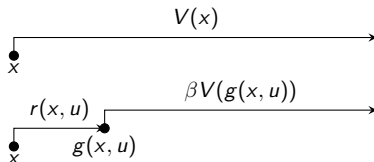
Bellman Principle (Bellman, 1957, p. 83)

“An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.”

Bellman Equation

For each given $x \in \mathbb{R}^n$, the value function V satisfies:

$$V(x) = \max_{u \in \mathbb{R}^k} r(x, u) + \beta V(g(x, u)).$$



Bellman Principle of Optimality

Problem

Find a function V that satisfies that, for each given $x \in \mathbb{R}^n$,

$$V(x) = \underbrace{\max_{u \in \mathbb{R}^k} r(x, u) + \beta V(g(x, u))}_{=: T(V)(x)}.$$

- For any candidate value function V , let $T(V)$ be the RHS of the Bellman equation:

$$T(V)(x) = \max_{u \in \mathbb{R}^k} r(x, u) + \beta V(g(x, u)) \text{ for each } x.$$

- We want to find a fixed point of T : namely, V such that

$$V = T(V).$$

Bellman Principle of Optimality

Problem

Find a function V that satisfies that, for each given $x \in \mathbb{R}^n$,

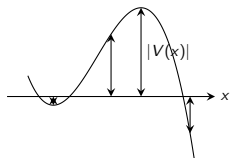
$$V(x) = \underbrace{\max_{u \in \mathbb{R}^k} r(x, u) + \beta V(g(x, u))}_{=: T(V)(x)}.$$

- The operator T (which maps a candidate value function to another value function) is a *contraction* with modulus β .
- This means:
 - ▶ T has a *unique* fixed point, i.e., the value function exists and is unique.
 - ▶ For any initial candidate value function V_0 , the value function can be obtained as the limit of $V_n := T^n V_0$.

Solving a Bellman Equation

- Let $X = B(\mathbb{R}^n, \mathbb{R})$ be the set of bounded continuous functions $V : \mathbb{R}^n \rightarrow \mathbb{R}$.
- The space $B(\mathbb{R}^n, \mathbb{R})$ is a complete normed space, endowed with $\|\cdot\| : B(\mathbb{R}^n, \mathbb{R}) \rightarrow \mathbb{R}$ defined by

$$\|V\| := \sup_{x \in \mathbb{R}^n} |V(x)|.$$



Contraction Mapping Theorem

- An operator $T : X \rightarrow X$ is a contraction with modulus $\beta \in (0, 1)$ if

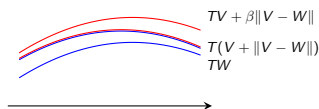
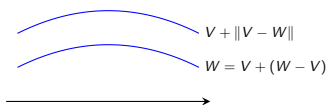
$$\|TV - TW\| \leq \beta \|V - W\| \text{ for all } V, W \in X.$$

- The contraction mapping theorem:

- 1 Any contraction $T : X \rightarrow X$ has a unique fixed point $V^* \in X$, i.e., $TV^* = V^*$.
- 2 The unique fixed point is obtained by $\lim_{n \rightarrow \infty} T^n V_0$ for any given $V_0 \in X$.

Solving a Bellman Equation

- Why is T a contraction mapping?
- $\Leftarrow$ Blackwell condition



- Monotonicity: if $V \leq W$ then

$$\begin{aligned}TV(x) &= \max_u r(x, u) + \beta V(g(x, u)) \\ &\leq \max_u r(x, u) + \beta W(g(x, u)) = TW(x) \text{ for all } x.\end{aligned}$$

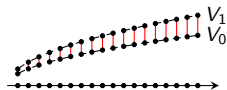
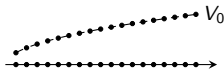
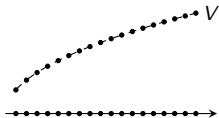
- Discounting:

$$\begin{aligned}T(V + \alpha)(x) &= \max_u r(x, u) + \beta(V + \alpha)(g(x, u)) \\ &= \max_u r(x, u) + \beta V(g(x, u)) + \beta\alpha \\ &= T(V)(x) + \beta\alpha \text{ for all } x.\end{aligned}$$

Solving a Bellman Equation

Value Function Iteration

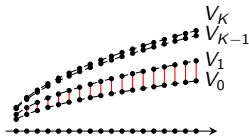
- 1 ▶ We approximate the value function V as a vector $(V(x))_x$ of values defined on a grid of state variables.
▶ Thus, create a grid of state variables.
- 2 Choose an initial guess (vector) V_0 of the value function.
- 3 Numerically solve the maximization problem:
 $V_1(x) = TV_0(x) = \max_u r(x, u) + \beta V_0(g(x, u))$ for each node x .
Then we have V_1 .
- 4 If $\|V_1 - V_0\| = \max_x |V_1(x) - V_0(x)|$ is less than a predetermined tolerance level $\varepsilon > 0$, then terminate the algorithm (with the solution V_1). If not, iterate the procedure.



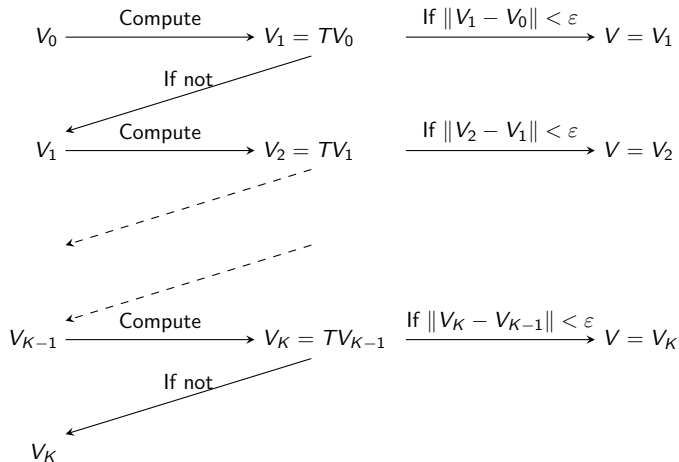
Solving a Bellman Equation

Value Function Iteration

- ➊ We approximate the value function V as a vector $(V(x))_x$ of values defined on a grid of state variables.
 - Thus, create a grid of state variables.
- ➋ Choose an initial guess (vector) V_0 of the value function.
- ➌ Numerically solve the maximization problem:
 $V_1(x) = TV_0(x) = \max_u r(x, u) + \beta V_0(g(x, u))$ for each node x . Then we have V_1 .
- ➍ If $\|V_1 - V_0\| = \max_x |V_1(x) - V_0(x)|$ is less than a predetermined tolerance level $\varepsilon > 0$, then terminate the algorithm (with the solution V_1). If not, iterate the procedure.
- ➎
 - At the K -th iteration, starting with V_{K-1} ,
 - Compute
 $V_K(x) = \max_u r(x, u) + \beta V_{K-1}(g(x, u))$
for each x .
 - If $\|V_K - V_{K-1}\| < \varepsilon$, then finish the algorithm (with the solution V_K).
 - If not, iterate the procedure with V_K .
 - $\|V_K - V_{K-1}\| \leq \beta \|V_{K-1} - V_{K-2}\| \leq \dots \leq \beta^{K-1} \|V_1 - V_0\| \rightarrow 0$.



Solving a Bellman Equation



Solving a Bellman Equation

- Bellman equation (for ease of exposition, assume $n = k = 1$):

$$V(x) = \max_u r(x, u) + \beta V(g(x, u)).$$

- FOC: inter-temporal trade-off

$$\frac{\partial r(x, u^*)}{\partial u} + \beta V'(g(x, u^*)) \frac{\partial g(x, u^*)}{\partial u} = 0.$$

► What is $V'(g(x, u^*)) = V'(x')$?

- Envelope Formula: the marginal valuation with respect to state

$$V'(x) = \underbrace{\frac{\partial r(x, u^*)}{\partial x} + \beta V'(g(x, u^*)) \frac{\partial g(x, u^*)}{\partial x}}_{\text{partial derivative of the R.H.S. w.r.t. } x}.$$

- We will derive the Euler equation from these two conditions in particular contexts (when r depends only on u).

Solving a Bellman Equation

Derivation of the Envelope Formula (Optional)

- Letting h be the policy function:

$$V(x) = r(x, h(x)) + \beta V(g(x, h(x))).$$

- Differentiating both sides with respect to x :

$$\begin{aligned} V'(x) &= \frac{\partial}{\partial x} r(x, h(x)) + \frac{\partial}{\partial u} r(x, h(x)) h'(x) + \beta V'(g(x, h(x))) \frac{\partial}{\partial x} g(x, h(x)) + \beta V'(g(x, h(x))) \frac{\partial}{\partial u} g(x, h(x)) h'(x) \\ &= \underbrace{\left\{ \frac{\partial}{\partial u} r(x, h(x)) + \beta V'(g(x, h(x))) \frac{\partial}{\partial u} g(x, h(x)) \right\}}_{=0: \text{ FOC}} h'(x) + \frac{\partial}{\partial x} r(x, h(x)) + \beta V'(g(x, h(x))) \frac{\partial}{\partial x} g(x, h(x)) \\ &= \frac{\partial}{\partial x} r(x, h(x)) + \beta V'(g(x, h(x))) \frac{\partial}{\partial x} g(x, h(x)) \\ &= \frac{\partial}{\partial x} r(x, u^*) + \beta V'(g(x, u^*)) \frac{\partial}{\partial x} g(x, u^*). \end{aligned}$$

Cake-Eating Problem

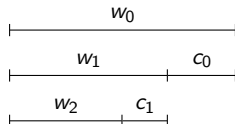
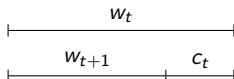
Cake Eating Problem

- 1 At the beginning of time 0, an agent is endowed with a cake of size $w_0 > 0$.
- 2 The problem is to find an optimal consumption plan $(c_t)_{t=0}^{\infty}$, where c_t is consumption at time t .
- 3 The agent's period-by-period utility function:

$$u(c_t) = \log(c_t).$$

Not dependent on the size of the cake left at each period.

- 4 The evolution of cake over time:



Cake Eating Problem

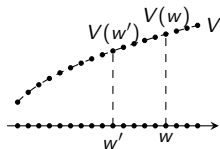
There are multiple ways to assign state/control variables.

- ①
 - ▶ A state variable: the size of the cake left at each period: $x_t = w_t$ for each $t \in \mathbb{N}_0$.
 - ▶ A control variable: consumption at each period: $u_t = c_t$ for each t .
 - ▶ The transition equation: $x_{t+1} = g(x_t, u_t) = x_t - u_t$ for each t .
- ⇒ The Bellman equation:

$$V(w) = \max_{c \in [0, w]} \log(c) + \beta V(\underbrace{w - c}_{\text{may not be on the grid}}).$$

- ①
 - ▶ A state variable: the size of the cake left at each period: $x_t = w_t$ for each $t \in \mathbb{N}_0$.
 - ▶ A control variable: the size of the cake left at the next period: $u_t = w_{t+1}$ for each t .
 - ▶ The transition equation: $x_{t+1} = u_t$ for each t .
- ⇒ The Bellman equation:

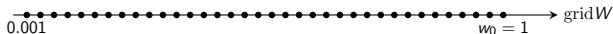
$$V(w) = \max_{w' \in [0, w]} \log(w - w') + \beta V(w').$$



Cake Eating Problem

- 1 We approximate the value function V as a vector $(V(w))_w$ of values defined on a discrete grid of the interval $(0, w_0]$.
 - ▶ We consider a uniform grid on possible cake sizes.
 - ▶ In the code, `gridW` is a $1 \times \text{dimW}(= 1 \times 2000)$ matrix (vector) starting from 0.001 (in order to avoid $\log(0) = -\text{inf}$) to $w_0 = 1$:

$$\text{gridW} = \left[\underbrace{\text{gridW}(1)}_{=0.001} \quad \text{gridW}(2) \quad \dots \quad \dots \quad \underbrace{\text{gridW}(\text{dimW})}_{=w_0=1} \right].$$



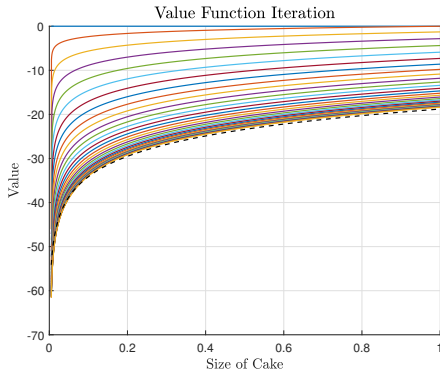
- 2 It's convenient to use the second formulation of the Bellman equation:

$$V(w) = \max_{w' \in [0, w]} \log(w - w') + \beta V(w').$$

- ▶ In the first formulation, we may need to interpolate the value of V at $w - c$ if it is not on the grid.

Cake Eating Problem

- ③ Choose an initial guess (vector) V_0 of the value function.
- ▶ In the code, let $V_0(w) = 0$ for every node w in the grid.
 - ▶ Generally, if we knew a better guess $\Rightarrow$ faster convergence.
 - ▶ (Note) Since it can be seen that $V \leq V_1 \leq V_0$, $V_n \downarrow V$.



Cake Eating Problem

- ④ We numerically solve: for each node w in the grid,

$$V_1(w) = \overbrace{\max_{w' \in [0, w]} \log(w - w') + \beta V_0(w')}^{TV_0(w)}.$$

- ▶ Then we have the value function (vector) V_1 .
- ▶ MATLAB is slower in for-loops $\Rightarrow$ use matrices.

$w = \text{gridW}(i)$: today (row i)

$w' = \text{gridW}(j)$: tomorrow (column j)

$\max_j \Rightarrow (\text{dimW}, 1)$



$$\begin{array}{c} \text{today} \\ \text{(rows } i) \\ \longrightarrow \end{array} \overbrace{\left(V_1(\underbrace{\text{gridW}(i)}_{=w}) \right)}^{(\text{dimW}, 1)} = \underbrace{\max_j \left(\log \left(\underbrace{\text{gridW}(i) - \text{gridW}(j)}_{=w - w'} \right) \right)}_{(\text{dimW}, \text{dimW})} + \beta \underbrace{\left(V_0(\underbrace{\text{gridW}(j)}_{=w'}) \right)}_{(\text{dimW}, \text{dimW})}$$

Cake Eating Problem

- 5 If $\|V_1 - V_0\| = \max_w |V_1(w) - V_0(w)|$ is less than a predetermined tolerance level ε , then terminate the algorithm.
If not, iterate the procedure with V_1 .

► In the codes,

$$\text{norm}(V_1 - V_0, \text{Inf}) = \max_{w \in \text{gridW}} |V_1(w) - V_0(w)|.$$

- 6 The policy function is $c = w - w'$, that is,

$$h(\text{gridW}(i)) = \text{gridW}(i) - \text{gridW}(j(i)),$$

where $j(i)$ solves the RHS of the Bellman equation when the state is $\text{gridW}(i)$.

Remarks on the MATLAB Codes

- We construct a matrix of possible consumption for each combination of the current and next period's cake sizes.
- If a current cake size is $\text{gridW}(i)$ and the next period's cake size is $\text{gridW}(j)$, the consumption in the current period is

$$\text{gridW}(i) - \text{gridW}(j).$$

- So, we construct a $\text{dimW} \times \text{dimW}$ matrix consumption, where its (i, j) element is

$$\text{consumption}(i, j) = \text{gridW}(i) - \text{gridW}(j).$$

- Because an (i, j) element with $i \leq j$ is not feasible (non-positive consumption!), we replace these elements with a value sufficiently close to zero (again, due to the log utility).

Remarks on the MATLAB Codes

MATLAB is suitable for matrix computations. So, instead of using for-loops, we construct the following two auxiliary matrices.

- The first is the $\text{dimW} \times \text{dimW}$ matrix whose i -th row is

$$[\text{gridW}(i) \quad \text{gridW}(i) \quad \dots \quad \dots \quad \text{gridW}(i)] .$$

- The second is the $\text{dimW} \times \text{dimW}$ matrix whose j -th column is

$$[\text{gridW}(j) \quad \text{gridW}(j) \quad \dots \quad \dots \quad \text{gridW}(j)]' = \begin{bmatrix} \text{gridW}(j) \\ \text{gridW}(j) \\ \vdots \\ \vdots \\ \text{gridW}(j) \end{bmatrix} .$$

- The consumption matrix is the difference of these two matrices.

Remarks on the MATLAB Codes

The first auxiliary matrix is constructed in the following way:

$$\begin{aligned} \text{gridW}' * \text{ones}(1, \text{dimW}) &= \begin{bmatrix} \text{gridW}(1) \\ \text{gridW}(2) \\ \vdots \\ \vdots \\ \text{gridW}(\text{dimW}) \end{bmatrix} \begin{bmatrix} 1 & 1 & \dots & \dots & 1 \end{bmatrix} \\ &= \begin{bmatrix} \text{gridW}(1) & \text{gridW}(1) & \dots & \dots & \text{gridW}(1) \\ \text{gridW}(2) & \text{gridW}(2) & \dots & \dots & \text{gridW}(2) \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ \text{gridW}(\text{dimW}) & \text{gridW}(\text{dimW}) & \dots & \dots & \text{gridW}(\text{dimW}) \end{bmatrix}. \end{aligned}$$

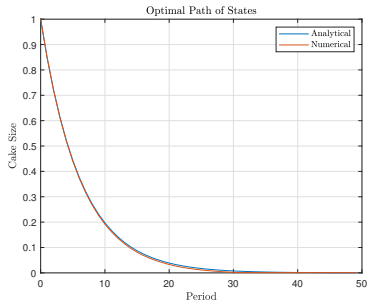
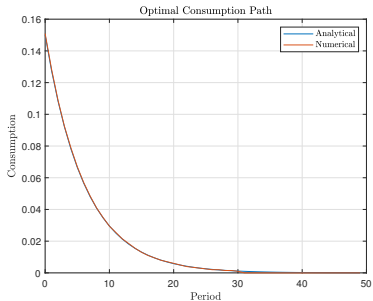
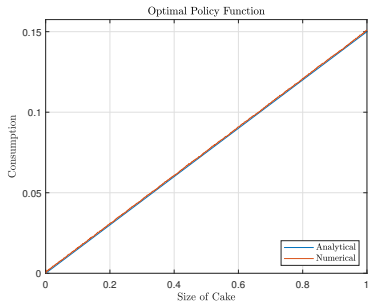
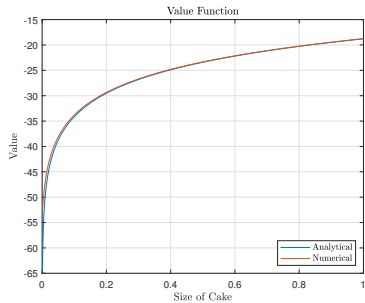
Remarks on the MATLAB Codes

The second auxiliary matrix is constructed in the following way:

$$\begin{aligned} \text{ones}(\text{dimW}, 1) * \text{gridW} &= \begin{bmatrix} 1 \\ 1 \\ \vdots \\ \vdots \\ \vdots \\ 1 \end{bmatrix} [\text{gridW}(1) \quad \text{gridW}(2) \quad \dots \quad \dots \quad \text{gridW}(\text{dimW})] \\ &= \begin{bmatrix} \text{gridW}(1) & \text{gridW}(2) & \dots & \dots & \text{gridW}(\text{dimW}) \\ \text{gridW}(1) & \text{gridW}(2) & \dots & \dots & \text{gridW}(\text{dimW}) \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ \text{gridW}(1) & \text{gridW}(2) & \dots & \dots & \text{gridW}(\text{dimW}) \end{bmatrix}. \end{aligned}$$

Now it's time to simulate the cake-eating problem with MATLAB!

Cake Eating Problem



Closed-Form Solution

- Unlike many other dynamic programming problems, the cake-eating problem with log (or CRRA) utility admits a closed-form solution.
- We can guess and verify that the value function takes a form of $V(w) = A + B \log(w)$ and then we can solve for A and B . We can then solve for the optimal policy function.
- We derive the closed-form solution by looking at the economic trade-off in this cake-eating problem through the first-order condition and the envelope formula.

Closed-Form Solution

Solving a Bellman Equation

- Bellman equation:

$$V(w) = \max_c u(c) + \beta V(w - c).$$

- FOC: inter-temporal trade-off

$$u'(c) = \beta \underbrace{V'(w - c)}_{w'}.$$

- Envelope Formula: the marginal valuation with respect to state

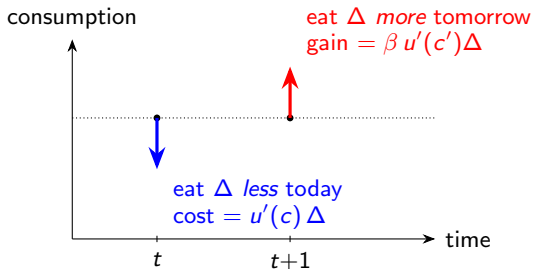
$$V'(w) = \beta \underbrace{V'(w - c)}_{=w'}.$$

- The Euler equation $u'(c) = \beta u'(c')$:

$$\begin{aligned} \overbrace{u'(c)} &= \beta \overbrace{V'(w')} && \text{(FOC)} \\ &= \beta \cdot \beta V'(w'') && \text{(Envelope)} \\ &= \underbrace{\beta u'(c')} && \text{(FOC)} \end{aligned}$$

Closed-Form Solution

Intuition behind the Euler equation (Optional)



- Eat Δ less today and more tomorrow:

$$\underbrace{u(c - \Delta)}_{=u(c) - u'(c)\Delta} + \underbrace{\beta u(c' + \Delta)}_{=\beta u(c') + \beta u'(c')\Delta}$$

- Two derivations:

- 1 Arbitrage: $u'(c)\Delta = \beta u'(c')\Delta \Rightarrow u'(c) = \beta u'(c')$.
- 2 FOC with respect to Δ at $\Delta = 0$: $u'(c) = \beta u'(c')$.

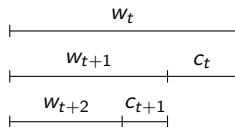
Closed-Form Solution

- The Euler Equation under log utility:

$$\begin{aligned}u'(c_t) &= \beta u'(c_{t+1}) \implies c_{t+1} = \beta c_t \\ &\implies c_\tau = \beta^{\tau-t} c_t.\end{aligned}$$

- The budget constraint and the optimality
 $\implies$ eat $(1 - \beta)$ of what's left:

$$\begin{aligned}w_t &= \sum_{\tau=t}^{\infty} c_\tau = \sum_{t=0}^{\infty} \beta^{\tau-t} c_t = \frac{1}{1-\beta} c_t \\ &\implies c_t = (1-\beta)w_t.\end{aligned}$$



- Then,

$$\begin{aligned}c_0 &= (1-\beta)w_0; \\ c_t &= \beta^t c_0 = (1-\beta)\beta^t w_0; \text{ and} \\ w_{t+1} &= w_t - c_t = \beta w_t.\end{aligned}$$

Closed-Form Solution

- Since $c_t = (1 - \beta)w_t$, the policy function is:

$$h(w) = (1 - \beta)w.$$

- Since $c_t = (1 - \beta)\beta^t w_0$, the value function is:

$$\begin{aligned} V(w) &= \sum_{t=0}^{\infty} \beta^t \log((1 - \beta)\beta^t w) \\ &= \sum_{t=0}^{\infty} \left(\beta^t \log(1 - \beta) + \overbrace{\beta^t \log(\beta^t)}^{t\beta^t \log(\beta)} + \beta^t \log(w) \right) \\ &= \underbrace{\frac{\log(1 - \beta)}{1 - \beta} + \frac{\beta}{(1 - \beta)^2} \log(\beta)}_A + \underbrace{\frac{1}{1 - \beta} \log(w)}_{B \log(w)}. \end{aligned}$$

- Note:

$$\begin{aligned} \sum_{t=0}^{\infty} t\beta^t &= \beta \sum_{t=0}^{\infty} t\beta^{t-1} = \beta \sum_{t=0}^{\infty} \frac{d}{d\beta} \beta^t = \beta \frac{d}{d\beta} \sum_{t=0}^{\infty} \beta^t \\ &= \beta \frac{d}{d\beta} \frac{1}{1 - \beta} = \beta \frac{1}{(1 - \beta)^2}. \end{aligned}$$

Ramsey Growth Model

Ramsey Growth Model

Sequential Problem

$$V(k_t) = \max_{(c_\tau)_{\tau=t}^{\infty}} \sum_{\tau=t}^{\infty} \beta^{\tau-t} u(c_\tau)$$

subject to $k_{\tau+1} = \underbrace{f(k_\tau)}_{Ak_\tau^\alpha} + (1 - \delta)k_\tau - c_\tau$ for each $\tau \geq t$

k_t given.

Ramsey Growth Model

Bellman Equation

$$V(k) = \max_c u(c) + \beta V(k')$$

subject to $k' = \underbrace{f(k)}_{Ak^\alpha} + (1 - \delta)k - c$ and k given.

① Consumption:

- ▶ The cake-eating problem: $c = k - k'$.
- ▶ The growth model: $c = f(k) + (1 - \delta)k - k'$.

② Capital

- ▶ The cake-eating problem: $(0, w_0]$, where w_0 : initial cake size.
- ▶ The growth model: $[k_{\min}, k_{\max}]$ containing the “steady-state” capital level k^* .

Ramsey Growth Model

- FOC:

$$u'(c_t) = \beta V'(k_{t+1}).$$

- Envelope Formula:

$$V'(k_t) = \beta V'(k_{t+1})(f'(k_t) + 1 - \delta).$$

⇒ The Euler Equation:

$$\begin{aligned} u'(c_t) &= \beta^2 V'(k_{t+2})(f'(k_{t+1}) + 1 - \delta) \\ &= \beta u'(c_{t+1})(f'(k_{t+1}) + 1 - \delta). \end{aligned}$$

Ramsey Growth Model

- The Euler equation:

$$\begin{aligned}u'(c_t) &= \beta^2 V'(k_{t+2})(f'(k_{t+1}) + 1 - \delta) \\ &= \beta u'(c_{t+1})(f'(k_{t+1}) + 1 - \delta).\end{aligned}$$

- The transition equation:

$$k_{t+1} = f(k_t) + (1 - \delta)k_t - c_t.$$

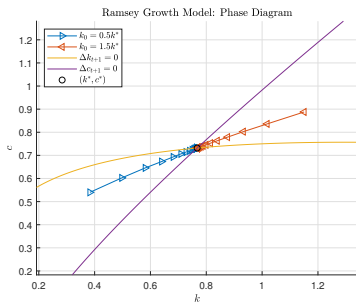
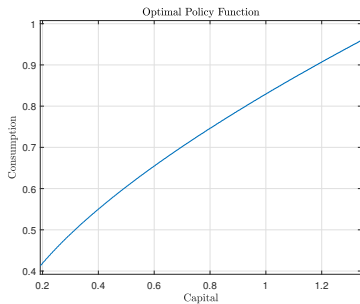
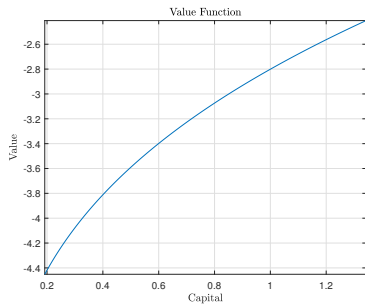
⇒ In the steady state in which $c_t = c_{t+1} = c^*$ and $k_t = k_{t+1} = k^*$,

$$\frac{1}{\beta} = \underbrace{f'(k^*)}_{=\alpha A(k^*)^{\alpha-1}} + 1 - \delta \implies k^* = \left(\frac{\alpha A}{\frac{1}{\beta} - (1 - \delta)} \right)^{\frac{1}{1-\alpha}}$$

and

$$\begin{aligned}c^* &= f(k^*) - \delta k^* \\ &= A(k^*)^\alpha - \delta k^*.\end{aligned}$$

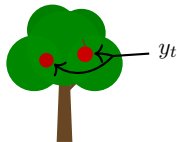
Ramsey Growth Model



Lucas Asset-Pricing Model

Lucas Asset-Pricing Model

- There are a large number of identical agents, modeled as a representative agent.
- An endowment economy with a single non-storable consumption good (“fruit”), which is given off stochastically by a single productive unit (“tree”).
- A consumption endowment (a sequence of consumption goods) $(y_t)_{t=0}^{\infty}$, where $y_t \geq 0$:
 - ▶ Markovian: $y_{t+1} = G(y_t, \varepsilon_{t+1})$ where $(\varepsilon_t)_{t=1}^{\infty}$ is an i.i.d. shock sequence with a known distribution ϕ .
 - ▶ Later on, $(y_t)_{t=0}^{\infty}$ follows:



$$\log y_{t+1} = \alpha \log y_t + \sigma \varepsilon_{t+1},$$

$$y_t \xrightarrow{\varepsilon_{t+1}} y_{t+1}$$

where $(\varepsilon_t)_{t=1}^{\infty}$ is i.i.d. and standard normal.

- Holding assets is the only way to transfer wealth into the future.
- “Lucas tree:” a claim on the consumption endowment.

Lucas Asset-Pricing Model

- The representative agent ranks consumption streams $(c_t)_{t=0}^{\infty}$ according to

$$\mathbb{E}_0 \left[\sum_{t=0}^{\infty} \beta^t u(c_t) \right],$$

where:

- $\beta \in (0, 1)$ is a fixed discount factor;
- u is a CRRA utility function:

$$u(c) = \begin{cases} \frac{c^{1-\gamma} - 1}{1-\gamma} & \text{if } \gamma > 0 \text{ and } \gamma \neq 1; \\ \log(c) & \text{if } \gamma = 1 \end{cases};$$

- $\mathbb{E}_0$ is the expectation operator (taken at time 0).

Pricing a Lucas Tree

Pricing an ex dividend claim:

- The seller retains this period's dividend.
- The buyer pays p_t today to purchase a claim on
 - ▶ y_{t+1} and
 - ▶ the right to sell the claim tomorrow at price p_{t+1} .

Solving a Competitive Equilibrium

- 1 Solving the consumer's problem, taking prices as given.
⇐ Dynamic Programming
- 2 Equilibrium constraints to back out equilibrium prices.

prices $\longrightarrow$ consumption

prices $\longleftarrow$ consumption c_t
||
endowment y_t

The Consumer's Problem: Set-up

$$\begin{aligned} \max_{(c_t)_{t=0}^{\infty}, (\pi_t)_{t=1}^{\infty}} \mathbb{E}_0 \left[\sum_{t=0}^{\infty} \beta^t u(c_t) \right] \\ \text{subject to } c_t + p_t \pi_{t+1} \leq \underbrace{(y_t + p_t) \pi_t}_{\text{payoff}} \\ c_t \geq 0, 0 \leq \pi_t \leq 1. \end{aligned}$$

- The decision to hold share π_t is made at time $t - 1$.
- ⇒ Inherited as a state variable at time t (π_0 given).

- Prices depend on current information.

- Current information is the endowment process up until the current period.

- However, the endowment process is Markovian.

⇒ The only relevant information is the current state $y (= y_t) \geq 0$.

⇒ We guess and verify an equilibrium where price p is a function of y .

$$y_t \xrightarrow{\varepsilon_{t+1}} y_{t+1}$$

$$y \xrightarrow{z'} y'$$

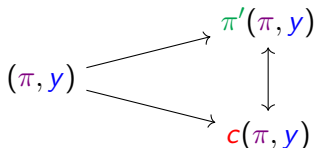
The Consumer's Problem: Dynamic Programming

- Bellman equation:

$$v(\pi, y) = \max_{\mathbf{c}, \pi'} u(\mathbf{c}) + \beta \underbrace{\int v(\pi', G(y, z)) \phi(dz)}_{=\mathbb{E}[v(\pi', G(y, z))]}$$

subject to $\mathbf{c} + p(y)\pi' \leq \underbrace{(y + p(y))\pi}_{\text{payoff}}.$

- Note: the consumer is a price-taker.
- The budget constraint is binding $\Rightarrow \mathbf{c} = (y + p(y))\pi - p(y)\pi'.$

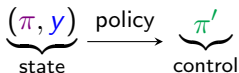


The Consumer's Problem: Dynamic Programming

- Bellman equation:

$$v(\pi, y) = \max_{\pi'} u(\underbrace{(y + p(y))\pi - p(y)\pi'}_{=c}) + \beta \underbrace{\int v(\pi', G(y, z))\phi(dz)}_{=\mathbb{E}[v(\pi', G(y, z))]}.$$

- The solution to this dynamic programming problem is an optimal policy π' as a function of the state variable (π, y) .



Equilibrium Constraints

Solving a Competitive Equilibrium

- 1 Solving the consumer's problem,
taking prices as given.

⇐ Dynamic Programming ✓

$$(p_t)_t \longrightarrow \begin{pmatrix} (c_t)_t \\ (\pi_t)_t \end{pmatrix}$$

- 2 Equilibrium constraints to back out equilibrium prices.

- 1 The consumption good is non-storable.
⇒ In equilibrium, we must have $c_t = y_t$ for all t .

- 2 The representative consumer
(or all consumers are identical).
⇒ There should be no trade in equilibrium.
⇒ The representative consumer owns the whole
tree in every period: $\pi_t = 1$ for all t .

$$(p_t)_t \longleftarrow \begin{pmatrix} c_t = y_t \\ \pi_t = 1 \end{pmatrix}$$

⇒ Prices have to adjust so that $c_t = y_t$ and $\pi_t = 1$ for all t .

The Equilibrium Pricing Function

Bellman Equation

$$v(\pi, y) = \max_{\pi'} u(\underbrace{(y + p(y))\pi - p(y)\pi'}_{=c}) + \beta \int v(\pi', G(y, z))\phi(dz).$$

- The first-order condition (w.r.t. π'):

$$u'(c)p(y) = \beta \int v'_1(\pi', G(y, z))\phi(dz), \text{ where } v'_1 = \frac{\partial v}{\partial \pi'}.$$

- What is v'_1 ? (Envelope Formula)

- ▶ Differentiate both sides w.r.t. $\pi \Rightarrow$

$$v'_1(\pi, y) = u'(c)(y + p(y)).$$

The Equilibrium Pricing Function

- The first-order condition (condition on a control variable π'):

$$u'(c)p(y) = \beta \int v'_1(\pi', G(y, z))\phi(dz).$$

- The envelope condition (condition on a state variable π):

$$v'_1(\pi, y) = u'(c)(y + p(y)).$$

- Equilibrium conditions: $c = y$ and $\pi = \pi' = 1$.

$$\Rightarrow \underbrace{u'(y)p(y)}_{\text{cost of increasing tomorrow's share}} = \beta \int \underbrace{u'(G(y, z))(G(y, z) + p(G(y, z)))}_{\text{gain from increasing tomorrow's share}} \phi(dz).$$

The Equilibrium Pricing Function

Consumption-based Asset Pricing Equation

- Functional equation:

$$p(y) = \beta \int \frac{u'(G(y, z))}{u'(y)} (G(y, z) + p(G(y, z))) \phi(dz).$$

- ▶ The functional equation in the unknown function p .
- ▶ The solution is an equilibrium price vector p^* .

- (Slight digression) Sequential form:

$$p_t = \mathbb{E}_t \left[\underbrace{\beta \frac{u'(y_{t+1})}{u'(y_t)}}_{\text{stochastic discount factor}} \underbrace{(y_{t+1} + p_{t+1})}_{\text{payoff}} \right].$$

Solving the Model

Functional Equation

$$p(y) = \beta \int \frac{u'(G(y, z))}{u'(y)} (G(y, z) + p(G(y, z))) \phi(dz).$$

(Lucas') Indirect Approach

$$\begin{aligned} u'(y)p(y) &= \beta \int u'(G(y, z))(G(y, z) + p(G(y, z)))\phi(dz) \\ &= \underbrace{\beta \int u'(G(y, z))G(y, z)\phi(dz)}_{h(y)} + \beta \int u'(G(y, z))p(G(y, z))\phi(dz). \end{aligned}$$

Solving the Model

(Lucas') Indirect Approach

$$\underbrace{u'(\textcolor{red}{y})\textcolor{red}{p}(\textcolor{red}{y})}_{\textcolor{teal}{f}(\textcolor{red}{y})} = \beta \int u'(G(y, z))(G(y, z) + p(G(y, z)))\phi(dz)$$
$$= \beta \underbrace{\int u'(G(y, z))G(y, z)\phi(dz)}_{h(y)} + \beta \int \underbrace{u'(\textcolor{red}{G}(y, z))\textcolor{red}{p}(\textcolor{red}{G}(y, z))}_{\textcolor{teal}{f}(\textcolor{red}{G}(y, z))}\phi(dz).$$

Functional Equation

$$\textcolor{teal}{f}(y) = h(y) + \beta \int \textcolor{teal}{f}(G(y, z))\phi(dz),$$

where

$$f(y) := u'(y)p(y) \text{ and } h(y) := \beta \int u'(G(y, z))G(y, z)\phi(dz).$$

Solving the Model

Functional Equation

$$f(y) = h(y) + \underbrace{\beta \int f(G(y, z)) \phi(dz)}_{=:(Tf)(y)}.$$

- We want to find a fixed point $f = f^*$ of T :

$$f^* = Tf^*, \text{ i.e., } f^*(y) = Tf^*(y) \text{ for all } y.$$

Solving the Model

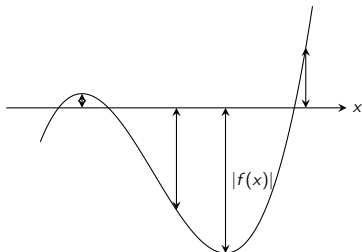
Mapping T

- Recall:

$$(Tf)(y) := h(y) + \beta \int f(G(y, z))\phi(dz).$$

- T maps any continuous bounded function $f : \mathbb{R}_+ \rightarrow \mathbb{R}$ to a new continuous and bounded function $Tf : \mathbb{R}_+ \rightarrow \mathbb{R}$.
- Formally, let $B(\mathbb{R}_+, \mathbb{R})$ be the set of bounded and continuous functions on $\mathbb{R}_+$.
- Then, $T : B(\mathbb{R}_+, \mathbb{R}) \rightarrow B(\mathbb{R}_+, \mathbb{R})$.
- The space $B(\mathbb{R}_+, \mathbb{R})$ is a complete normed space, where

$$\|f\| := \sup_{y \in \mathbb{R}_+} |f(y)|.$$



Solving the Model

Mapping T is a contraction with modulus β

Proof. Take $f, g \in B(\mathbb{R}_+, \mathbb{R})$. For any $y \in \mathbb{R}_+$, we have:

$$\begin{aligned} |(Tf)(y) - (Tg)(y)| &= \left| \beta \int f(G(y, z))\phi(dz) - \beta \int g(G(y, z))\phi(dz) \right| \\ &= \beta \left| \int f(G(y, z))\phi(dz) - \int g(G(y, z))\phi(dz) \right| \\ &\leq \beta \int |f(G(y, z)) - g(G(y, z))|\phi(dz) \\ &\leq \beta \int \|f - g\|\phi(dz) \\ &= \beta \|f - g\|. \end{aligned}$$

Taking the supremum, we obtain

$$\|Tf - Tg\| \leq \beta \|f - g\|,$$

as desired.

Numerical Computation

- We compute f^* by iterating T by picking an arbitrary f .
- The equilibrium price function p^* is then recovered from

$$p^*(y) = \frac{f^*(y)}{u'(y)}.$$

Special Case: log utility

- If $u(c) = \log(c)$, then

$$h(y) = \beta \int u'(G(y, z)) G(y, z) \phi(dz) = \beta \int \phi(dz) = \beta.$$

- The functional equation:

$$f(y) = \beta + \beta \int f(G(y, z)) \phi(dz).$$

- Constant f ?

$$\begin{aligned} f &= \beta + \beta f \Rightarrow f = \frac{\beta}{1 - \beta} \\ &\Rightarrow p(y) = \frac{\beta}{1 - \beta} y. \end{aligned}$$

Numerical Computation

- Define a grid on $y \in [0, \infty)$.
- Since $\log y_{t+1} = \alpha \log y_t + \sigma \varepsilon_{t+1}$,

$$\mathbb{E}[\log y_t] = 0 \text{ and } \mathbb{V}[\log y_t] = \frac{\sigma^2}{1 - \alpha^2}.$$

- A uniform grid:

$$y_vec = [y_vec(1) \quad \cdots \quad \cdots \quad y_vec(100)] ,$$

where

$$y_vec(1) = y_min = \exp \left(-4 \cdot \frac{\sigma}{\sqrt{1 - \alpha^2}} \right) \text{ and}$$

$$y_vec(100) = y_max = \exp \left(4 \cdot \frac{\sigma}{\sqrt{1 - \alpha^2}} \right) .$$

Numerical Computation

- Letting $r_t = \exp(\sigma \varepsilon_t)$,

$$y_{t+1} = y_t^\alpha r_{t+1},$$

where $(r_t)_{t=1}^\infty$ is i.i.d. log normal: $\log(r_t) \sim N(0, \sigma^2)$.

- 500 seeds, with initialization: `rng('default')`.

- The integral

$$\begin{aligned} h(y) &= \beta \int u'(G(y, z)) G(y, z) \phi(dz) \\ &= \beta \int (G(y, z))^{1-\gamma} \phi(dz) \end{aligned}$$

at $y = y_vec(i)$ is approximated by the average of

$$\beta ((y_vec(i))^\alpha \cdot r)^{1-\gamma},$$

where we consider 500 random seeds of log-normally distributed r .

Numerical Computation

- Similarly, the function Tf , given by

$$Tf(y) = \beta \int f(G(y, z)) \phi(dz) \text{ for each } y,$$

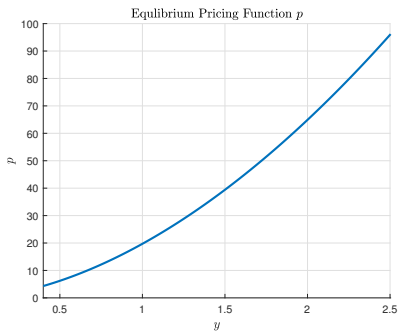
is approximated at $y = y_vec(i)$ by the average of

$$\beta f((y_vec(i))^\alpha \cdot r).$$

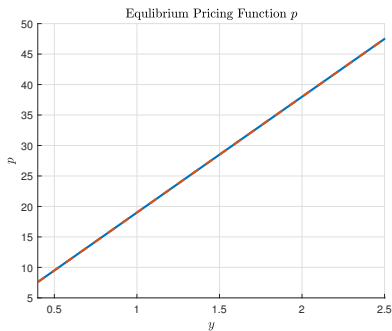
- The problem is that, when we consider a vector $f = (f(y_vec(i)))_i$ on the grid, f may not be defined at the point $(y_vec(i))^\alpha \cdot r$.
- In the code,

$$\beta \cdot \text{interp1}(\underbrace{y_vec, f}_{\text{original data points}}, \underbrace{(y_vec(i))^\alpha \cdot r}_{\text{approximate } f \text{ on these points}}, 'spline').$$

Numerical Computation



$$u(c) = \frac{c^{1-\gamma} - 1}{1-\gamma} \text{ with } \gamma = 2$$



$$u(c) = \log(c)$$
$$p(y) = \frac{\beta}{1-\beta}y$$

References

Books

- ① Adda, Jérôme and Russell Cooper. (2003) *Dynamic Economics: Quantitative Methods and Applications*. The MIT Press.
- ② Judd, Kenneth L. (1998) *Numerical Methods in Economics*. The MIT Press.
- ③ Ljungqvist, Lars and Thomas Sargent. (2018) *Recursive Macroeconomic Theory*. The Fourth Edition. The MIT Press.
- ④ Stokey, Nancy L., Robert E. Lucas, and Edward E. Prescott. (1989) *Recursive Methods in Economic Dynamics*. Harvard University Press.

Online Resources

- ① Sigmon, Kermit. MATLAB Primer. (also the latest version as a book)
- ② Perla, Jesse and Sargent, Thomas J., and Stachurski, John. Quantitative Economics with Julia/Advanced Quantitative Economics with Python (Asset Pricing II: The Lucas Asset Pricing Model)
https://julia.quantecon.org/multi_agent_models/lucas_model.html
https://python-advanced.quantecon.org/lucas_model.html